Ders 3 Genişletilmiş Notu: Basit Sıralama Algoritmaları ve Karmaşıklık Analizi

Ders 3 Genişletilmiş Notu: Basit Sıralama Algoritmaları ve Karmaşıklık Analizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kaynak Kitapları
- 2. İyi Bir Algoritmanın Beş Hedefi
- 3. Bubble Sort (Kabarcık Sıralaması)
- 4. Selection Sort (Seçmeli Sıralama)
- 5. Insertion Sort (Araya Eklemeli Sıralama)
- 6. Shell Sort
- 7. Karmaşıklık Nedir?
- 8. Algoritmaların Karmaşıklık Hesabı
- 9. Karmaşıklığın Çalışma Süresine Etkisi
- 10. Yerinde Sıralama (In-place Sorting)
- 11. Kararlı Sıralama (Stable Sorting)

1. Dersin Kaynak Kitapları

Hoca derse, önceki hafta eksik kaldığını düşündüğü kitap listesini tamamlayarak başladı. Üç kitap kullanıyor:

- *Algorithms, 4th Edition* (Robert Sedgewick, Kevin Wayne): en çok kullandığı kitap.
- *Introduction to the Design & Analysis of Algorithms* (Anany Levitin): ikinci en çok kullandığı kitap.
- *The Algorithm Design Manual* (Steven S. Skiena): yan kaynak. Hocanın hatırladığına göre e-kitabı da var.

Bu kitaplar yalnız ders için değil, kendini geliştirmek için de okunabilir. Hoca bunu internetteki kaynaklarla karşılaştırarak gerekçelendirdi: İnternette çok fazla kaynak var ve çoğu kontrolsüz. İsteyen herkes bir şey yazabildiği için bazıları doğru olmayabilir, bazıları da verimli olmayabilir.

> **Hocanın vurgusu:** Güvenilir kaynaklardan bilgi edinmek çok önemlidir.

2. İyi Bir Algoritmanın Beş Hedefi

Bugünkü konu sıralama algoritmaları. Hoca, programlama derslerinde örnek olarak gösterilen temel algoritmalardan başlayacağını, ama bu kez sorunun “bir algoritmayı tasarlarken iyileştirme nasıl yapılır?” olduğunu söyledi. Veri yapıları ve algoritmalarda en önemli amaçlardan biri, olabildiğince verimli algoritmalar tasarlamaktır. Bunun için hoca, önceki hafta da söylediği beş hedefi hatırlattı:

- **Doğruluk:** Algoritma her türlü şartta ve uç noktalarda doğru çalışmalı.

- **Verimlilik:** Hem zamanı hem yeri iyi kullanmalı. Aynı amaç için birden fazla algoritma varsa, sizin problemin şartlarına en uygun olanı seçilmeli.
- **Kodlama kolaylığı:** Tasarım rahat kodlanabilir olmalı.
- **Sadelik:** "if kullanmayayım" diye şunu şundan çıkarıp ona bölmek her zaman istenen bir şey değildir. Çok ciddi bir zaman ya da yer kazancı sağlamıyorsa karmakarışık bir algoritma yazılmaz. Olabildiğince sade ve temiz yazılır.
- **Genellik:** Problem genel bir problemse, yalnız çift sayılara ya da yalnız 100'e kadar olan sayılara bakan bir çözüm doğru değildir. Sabit bilgiler için çalışan algoritma tasarlanmaz.

Verimlilik hedefinin nasıl yakalanacağı konusunda hoca önceki haftanın önerisini yineledi: Önce küçük bir veri grubu üzerinde deneme yaparak tasarlamak hataları azaltır. Örnek üzerinde çalışırken problemleri ve uç noktaları daha rahat görürsünüz. İlk seferde en iyi algoritmayı bulamayabilirsiniz. Önce kaba bir tasarım yapar, sonra "neleri azaltabilirim, daha iyi nasıl olur?" diye bakarsınız. Tabii ki istenen, en verimlisini ilk adımda görmektir, ama olmayabilir. Biraz emek harcanırsa birkaç adımda daha iyisini bulmak her zaman mümkündür. Hoca bu beş noktaya sırayla dikkat edildiğinde zamanla bir tarz oluştuğunu söyledi.

3. Bubble Sort (Kabarcık Sıralaması)

Sıralama algoritmalarının amacı, elimizdeki elemanları istenen ölçüte göre, küçükten büyüğe ya da büyükten küçüğe, sıralı hale getirmektir. Bunun için farklı algoritmalar geliştirilmiştir. Hoca, karmaşıklıkları birbirine benzeyen bu algoritmalardan hangisinin ne zaman tercih edileceğini de dersin sonunda değerlendireceğini söyledi.

Bubble Sort, "elimde n tane sayı var, sırala" denildiğinde algoritma tasarımına yeni başlayan kişilerin aklına ilk gelen algoritmadır.

3.1 Mantığı ve İlk Örnek

Algoritma ardışık elemanları ikişer ikişer karşılaştırır. Soldaki eleman sağdakinden büyükse, küçükten büyüğe sıraladığımız için yerlerini değiştirir. Örnek dizi: `6 2 11 7 5`.

Karşılaştırma	Sonuç	Dizi
6 ile 2	yer değişir	`2 6 11 7 5`
6 ile 11	değişmez	`2 6 11 7 5`
11 ile 7	yer değişir	`2 6 7 11 5`
11 ile 5	yer değişir	`2 6 7 5 11`

Birinci turun sonunda dizi `2 6 7 5 11` oldu. Hoca bu hal için ne söylenebileceğini sordu.

Soru: Bu turun sonunda ne elde ettik? **Cevap:** En büyük sayı en sona taşındı. Baştan ikişer ikişer karşılaştırınca en büyük sayı en sona taşınmış olur. Artık onunla işlem yapmaya gerek yoktur, yalnız ondan öncekilere bakılır.

İkinci turda `2 6 7 5` bölgesine bakılır. 2 ile 6, 6 ile 7 değişmez. 7 ile 5 yer değiştirir: `2 6 5 7 11`. Böylece 7 ve 11 yerleşmiş olur. Üçüncü turda 2 ile 6 değişmez, 6 ile 5 yer değiştirir: `2 5 6 7 11`. Dördüncü turda 2 ile 5 karşılaştırılır, değişmez. 2 tek kaldığı için karşılaştırılacak başka eleman yoktur, demek ki onun da yeri doğrudur.

Hoca burada bir noktaya dikkat çekti: Her turda yalnız bir eleman kesin olarak yerine yerleşir. Büyüklerin biraz ileri taşınması olur ama hep birlikte yerleşmezler, diğerleri karışık durur. Yerleşen tek eleman, bakılan bölgenin en büyüğüdür.

3.2 En Kötü Durum

Soru: En kötü durum ne olabilir? **Cevap (bir öğrenci):** Dizinin ters sıralı olması. Hoca bunu onayladı, çünkü büyükten küçüğe sıralı bir diziyi küçükten büyüğe çevirmeye çalışıyoruz. Örnek: `11 9 7 5 2`.

Tur	Yapılan yer değiştirmeler	Turun sonu	Yerleşen
1	11 sırayla 9, 7, 5, 2 ile değişir	`9 7 5 2 11`	11
2	9 sırayla 7, 5, 2 ile değişir	`7 5 2 9 11`	9
3	7 sırayla 5, 2 ile değişir	`5 2 7 9 11`	7
4	5 ile 2 değişir	`2 5 7 9 11`	5

5 eleman için 4 tur gerekti. Olabilecek en kötü durum budur. Genel olarak n eleman için $n-1$ tur yapılır, çünkü elemanlar ikişer ikişer karşılaştırılır ve son kalan eleman için ayrıca işlem gerekmez.

3.3 İlk Tasarım ve Sınırların Kontrolü

Hoca önce "düz mantıkla, üzerinde çok düşünmeden" bir tasarım yaptı. Sonra bunun nasıl iyileştirileceğini birlikte değerlendirdi. İlk tasarım şudur:

- n elemanlı `a` dizisi okunur.
- Dış döngü: $i = 1$'den $n-1$'e kadar (tur sayısı).
- İç döngü: $j = 0$'dan $n-i-1$'e kadar.
  - `a[j] > a[j+1]` ise `a[j]` ile `a[j+1]` yer değiştirir (swap). Değilse bir şey yapılmaz.
- Döngülerden çıkınca sıralı dizi yazdırılır.

Sınırları kontrol etmek için hoca `3 9 5 11 4` dizisini elle izledi. Burada $n = 5$.

$i = 1$ için en son $a[n-2]$ ile $a[n-1]$ karşılaştırılmalıdır. İç döngü $j = n-i-1 = n-2$'ye kadar gittiği için bu sağlanır. Tur boyunca 3 ile 9 değişmez (ara adım eklendi), 9 ile 5 yer değiştirir, 9 ile 11 değişmez, 11 ile 4 yer değiştirir: `3 5 9 4 11`.

$i = 2$ için en son $a[n-3]$ ile $a[n-2]$ karşılaştırılmak istenir, çünkü 11 yerleşti. $n-i-1 = n-3$ olduğundan bu da sağlanır. Tur sonunda 9 ile 4 yer değiştirir: `3 5 4 9 11`.

$i = 3$ için 3 ile 5 değişmez, 5 ile 4 yer değiştirir: `3 4 5 9 11`.

Son tur $i = n-1 = 4$ turudur. $n-i-1 = 5-4-1 = 0$ olur. İç döngü yalnız $j = 0$ için çalışır ve yalnız 0. ile 1. gözler, yani 3 ile 4 karşılaştırılır. Yer değişmez ve işlem biter.

> **Ders sonrası not:** Hoca bu izlemede $i = 3$ turunda yanlış yaptı: ikinci turun sonucu olan `3 5 4 9 11` dizisini üçüncü turun sonucu gibi yazdı. Bir öğrenci üçüncü turda 5'in yerleşmesi gerektiğini söyledi, hoca da düzeltti. Doğru ara adımlar: 1. tur `3 5 9 4 11`, 2. tur `3 5 4 9 11`, 3. tur `3 4 5 9 11`, 4. tur `3 4 5 9 11`.

Hoca bu uzun kontrolü, iki döngünün de bütün şartları sağlayıp sağlamadığını görmek için yaptığını söyledi. Dış döngü $n-1$ tur döner. İç döngünün her turda bir azalmasının sebebi, her adımda bakılan bölgenin en büyüğünün sona yerleşmesidir: 11 yerleştikten sonra ilk 4 elemana bakmak yeter.

İlk tasarımın tam C programı aşağıdadır. Dizi okumak yerine hocanın ilk örneği sabit olarak verilmiştir.

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}

void bubble_sort(int a[], int n) {
    int i, j;
    for (i = 1; i <= n - 1; i++)
        for (j = 0; j <= n - i - 1; j++)
            if (a[j] > a[j + 1])
                swap(&a[j], &a[j + 1]);
}
```

```c
int main(void) {
    int a[] = {6, 2, 11, 7, 5};
    int i, n = sizeof(a) / sizeof(a[0]);
    bubble_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Programın çıktısı `2 5 6 7 11` olur. Derste C kodu yazılmadı. Bu program, hocanın anlattığı tasarımın C karşılığıdır ve `gcc -Wall` ile derlenip çalıştırılmıştır.

3.4 İlk Tasarımın Sorunu

Sıkıntı şudur: Elemanlar yer değiştirse de değiştirmese de $n-1$ kez dönülür. Hoca küçükten büyüğe sıralı `2 3 5 9 11` dizisini verdi. $i = 1$ için j, 0'dan $n-2 = 3$'e kadar gider. 2 ile 3, 3 ile 5, 5 ile 9, 9 ile 11 karşılaştırılır, hiçbiri yer değiştirmez. Daha birinci turda kimse yer değiştirmedi. Yer değiştirme olmaması, yer değiştirmeye ihtiyaç olmaması, yani dizinin sıralı olması demektir. Buna rağmen algoritma körlemesine kontrole devam eder.

Soru: Bu program doğru çalışır mı? **Cevap:** Çalışır. Sorguladığımız şey doğru çalışmasıysa tasarım doğrudur. Peki verimli mi? Hayır.

> **Hocanın vurgusu:** Bu çok önemli. İşlem bitmiş olmasına rağmen devam etmek, çok farklı algoritmalarda sık yapılan bir hatadır. Algoritmanın çalışmaya devam etmesine gerek olup olmadığını kontrol etmelisiniz.

3.5 İyileştirilmiş Tasarım

Yer değiştirme yoksa, $n-1$ kez tekrar etmeye gerek kalmadan çıkılmalıdır. Bu, döngünün türünü değiştirir. Hoca önceki hafta konuşulan iki tür döngüyü hatırlattı: adım sayısı belli olan döngüler ve adım sayısı belli olmayan döngüler. Yapılan işin ne kadar süreceği belli değilse bu, adım sayısı belli olmayan bir döngüdür. `for` kullanıp `break` ile kırmayı da konuşmuşlardı: Tasarımı yaparken zaten bu döngünün adım sayısının belli olmadığını bilirsiniz. Buradaki durum da budur, çünkü yer değiştirme olmayana kadar gidilmesi gerekir. Bu yüzden `do-while` kullanılır.

İyileştirilmiş tasarım:

- $i = 1$ ile başlanır.
- Her turun başında yer değiştirme bayrağı (`swapped`) sıfırlanır.
- İç döngü aynıdır. Bir yer değiştirme olursa bayrak 1 yapılır, çünkü dizi hâlâ sıralanmaya devam ediyordur.
- İç döngüden çıkınca i artırılır.
- `swapped` ve $i < n$ şartlarının ikisi de sağlanıyorsa başa dönülür.

Hoca, bayrak dışında bir şartın daha gerekip gerekmediğini sordu. Bir öğrencinin cevabının ardından, dizi büyükten küçüğe sıralıysa son turda hâlâ yer değiştirme olacağını söyledi ve $i < n$ şartını ekledi. Dizi büyükten küçüğe sıralıysa en son $i = n-1$ için dönülür, o turda yer değişir. i'nin n olması artık karşılaştırmaya gerek olmadığı anlamına gelir.

> **Ders sonrası not:** $i < n$ şartı gereksiz bir boş turu önler, ama doğruluk için zorunlu değildir. Şart olmasa, ters sıralı dizide $i = n$ turunda iç döngü hiç çalışmaz, `swapped` 0 kalır ve döngü kendiliğinden biter. `11 9 7 5 2` dizisinde bu iki sürüm de 10 karşılaştırma yapar ve aynı sonucu verir.

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
```

```
}

void bubble_sort(int a[], int n) {
    int i = 1, j, swapped;
    do {
        swapped = 0;
        for (j = 0; j <= n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                swap(&a[j], &a[j + 1]);
                swapped = 1;
            }
        i++;
    } while (swapped && i < n);
}

int main(void) {
    int a[] = {2, 3, 5, 9, 11};
    int i, n = sizeof(a) / sizeof(a[0]);
    bubble_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Programın çıktısı `2 3 5 9 11` olur. Dizi sıralıysa birinci turda hiçbir şey yer değiştirmez ve algoritma hemen çıkar. Arada bir yerde dizi sıralanırsa orada çıkar. Hoca, Bubble Sort'un karmaşıklığının n^2 olduğunu söyledi ve ayrıntısını sonraya bıraktı.

4. Selection Sort (Seçmeli Sıralama)

4.1 Yer Değiştirme de Bir Maliyettir

Bubble Sort'un temel sıkıntısı, sıralama yapmak için durmadan yer değiştirmesidir. Hoca `6 5 3 7 8 9` gibi bir dizide sürekli ikişer karşılaştırıp gerekiyorsa yer değiştirildiğini hatırlattı. Gereksiz işleri yapmamak yalnız "elemanlar zaten sıralıysa hemen çık" demek değildir. O güzel bir çözümdü ama daha iyisi için rahatsız edici bir nokta daha vardır: Yer değiştirme de bir iş yüküdür. Elemanlara erişiyor, yerlerini değiştiriyorsunuz. Yapılan işlerin assembly karşılıklarını düşünürseniz bu önemli bir iştir.

Önceki dizide 11'i sona taşırken 5, 4 ve 3 ile tek tek yer değiştiririz. Selection Sort bu masrafı azaltmak için tasarlanmıştır. Fikir şudur: Bir tur dönüp bakılan bölgenin en küçüğü bulunur ve yalnız bir kez, o bölgenin en baştaki elemanıyla yer değiştirilir. Tur yine dönülür, ama yer değiştirme bir kez yapılır. Hoca, en büyüğü sona taşımanın da aynı şey olduğunu ekledi. Küçükten büyüğe sıralarken en küçüğü başa taşımak seçilmiştir.

Soru: Selection Sort nasıl çalışır? **Cevap (bir öğrenci):** Baştan sona tarar, en küçüğü bulur, en başa yapıştırır. Hoca "yapıştırır" ifadesinin karşılığını sordu ve netleştirdi: En küçük eleman, en baştaki elemanla yerini değiştirir.

4.2 Örnek

Dizi: `2 7 4 9 3 1 8`.

Tur (i)	Bakılan bölgenin en küçüğü	Yer değiştiren	Turun sonu
0	1 (5. gözde)	1 ile 2	1 7 4 9 3 2 8
1	2 (5. gözde)	2 ile 7	1 2 4 9 3 7 8
2	3 (4. gözde)	3 ile 4	1 2 3 9 4 7 8
3	4 (4. gözde)	4 ile 9 (ara adım eklendi)	1 2 3 4 9 7 8
4	7 (5. gözde)	7 ile 9	1 2 3 4 7 9 8
5	8 (6. gözde)	8 ile 9	1 2 3 4 7 8 9

Tur (i)	Bakılan bölgenin en küçüğü	Yer değiştiren	Turun sonu

İkinci turda 1 artık yerleştiği için ona hiç bakılmaz. En son turda $i = n - 2$ olur: $n - 2$. eleman 9'dur ve ondan sonra yalnız 8 vardır. İkisinin küçüğü bulunup yer değiştirilir. $n - 1$ için yapılacak bir şey kalmaz.

4.3 Sıralı Dizide Durma Şansı Var mı?

Soru: Dizi herhangi bir noktada küçükten büyüğe sıralanmış olsa da durma şansınız var mı? **Cevap (bir öğrenci):** Sanırım yok. Hoca bakarak gösterdi: Dizi sıralı halde olsa bile "en küçüğü bul, baştakiyle yerini değiştir" demek için bir tur dönmek zorundasınız, çünkü bakılan bölgenin en küçüğünün kim olduğunu bakmadan bilemezsiniz. Biri yerleştirdikten sonra yine aynısı gerekir. Dolayısıyla istediğiniz yerde kesme şansınız yoktur. Bütün dizi için bu bakma işlemi yapılmak zorundadır.

Avantajı ve dezavantajı:

- **Avantajı:** Yer değiştirmeleri en aza indirir.
- **Dezavantajı:** Dizi sıralı olsa da en küçük elemanı aramaya devam eder. Dizinin sıralı olduğunu fark edecek bir yapı yoktur.

4.4 Tasarım

Akış şöyledir:

- $i = 0$'dan $n - 2$'ye kadar:
  - Bakılan bölgenin en küçüğü başlangıçta ilk elemandır: `min = a[i]`, `adr = i`.
  - $j = i + 1$'den $n - 1$'e kadar: `a[j] < min` ise `min = a[j]`, `adr = j`.
  - `a[i]` ile `a[adr]` yer değiştirir.

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int t = *x;
    *x = *y;
    *y = t;
}

void selection_sort(int a[], int n) {
    int i, j, min, adr;
    for (i = 0; i <= n - 2; i++) {
        min = a[i];
        adr = i;
        for (j = i + 1; j <= n - 1; j++)
            if (a[j] < min) {
                min = a[j];
                adr = j;
            }
        swap(&a[i], &a[adr]);
    }
}

int main(void) {
    int a[] = {2, 7, 4, 9, 3, 1, 8};
    int i, n = sizeof(a) / sizeof(a[0]);
    selection_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
```

```
    return 0;
}
```

Programın çıktısı `1 2 3 4 7 8 9` olur.

4.5 Bubble Sort ile Karşılaştırma

Hoca, Bubble Sort'un en iyi durumdaki avantajını sordu. Dizi küçükten büyüğe sıralıysa Bubble Sort $n-1$ karşılaştırma yapar, yani mertebe olarak n. Selection Sort'ta daha iyisini yapma şansı yoktur. Dizinin küçükten büyüğe ya da büyükten küçüğe sıralı olması arasında fark yoktur, aynı işlemler yapılır. En iyi durumla en kötü durum Selection Sort'ta birbirine eşittir.

Yine de hocaya göre Selection Sort, genel anlamda Bubble Sort'tan daha iyidir. Çünkü bu işleri genelde ortalama durumda, sıralı olmayan sayılar üzerinde yaparız. Dizinin sıralı gelmesi çok beklediğimiz bir şey değildir.

5. Insertion Sort (Araya Eklemeli Sıralama)

5.1 Neden Önemli?

Bubble Sort ve Selection Sort'un en kötü durum karmaşıklığı n^2'dir. Insertion Sort, en kötü durumu n^2 olan algoritmalar içinde en verimli olanıdır ve küçük dizilerin sıralanması için tercih edilir.

Hoca bu bağlamda ileriki haftalara değindi. Dönemin ortasından sonra böl ve yönet (divide and conquer) algoritmalarında Quick Sort ve Merge Sort anlatılacak. Normalde en çok kullanılan sıralama algoritması, $n \log n$ karmaşıklığı nedeniyle Quick Sort'tur. Örneğin Java sıralama için Quick Sort kullanır. Fakat aynı Java, küçük dizi parçaları için Insertion Sort kullanır. Quick Sort özyinelemeli (recursive) bir algoritmadır ve küçük diziler üzerinde sürekli fonksiyon çağırmak yavaşlığa sebep olur. Bu yüzden küçük dizilerde karmaşıklığı n^2 olmasına rağmen en verimlisi Insertion Sort'tur. Hoca 20, 30, 40, 50 elemanlı diziler için Insertion Sort kullanmanın verimli bir çözüm olduğunu söyledi.

> **Ders sonrası not:** Hocanın Java için söylediği genel bir ifadedir. Java'da `int[]` gibi ilkel tipler için `Arrays.sort` büyük olasılıkla çift pivotlu Quick Sort kullanır ve küçük parçalarda Insertion Sort'a geçer. Nesne dizilerinde ise büyük olasılıkla kararlı bir Merge Sort türevi (TimSort) kullanılır.

5.2 Mantığı ve Örnek

Eleman, kendinden öncekilerle karşılaştırılır. Yerini bulduğu anda yerleşir ve durur. En başa kadar gitmesi de, her şeyi taraması da gerekmez. Mantık "yerini bulduğun zaman dur" üzerine kuruludur. Bu, yarı sıralı diziler için ve dizinin baştan sıralı olup arada yeni elemanların geldiği durumlar için çok avantajlıdır. O durumda yeni elemanı doğru yere yerleştirmek istersiniz.

Örnek dizi: `3 2 8 5 9 1`.

- 2, 3 ile karşılaştırılır. 2 küçük olduğu için 3 bir kaydırılır, başka eleman olmadığı için 2 başa yerleşir: `2 3 8 5 9 1`.
- 8, 3 ile karşılaştırılır. 8 büyük, geri kalanına bakmaya gerek yoktur. Baş taraf kendi içinde sıralı olduğu için 8, 3'ten büyükse 2'den de büyüktür. Hoca bunu algoritmanın güzel bir özelliği olarak andı.
- 5, 8 ile karşılaştırılır. 5 küçük, 8 bir kaydırılır. 5, 3 ile karşılaştırılır. 5 büyük, durur: `2 3 5 8 9 1`.
- 9, 8 ile karşılaştırılır, yer değişmez.
- 1 sırayla 9, 8, 5, 3, 2 ile karşılaştırılır. Hepsi büyük olduğu için hepsi bir kaydırılır. Karşılaştıracak eleman kalmayınca 1 başa yerleşir: `1 2 3 5 8 9`.

Hoca tasarıma geçmeden ara adımı bir kez daha yavaş gösterdi: Başı sıralı bir dizi olsun, `3 4 5 2 1`. 2 yerleştirilecek. 5 ile 2 karşılaştırılır, 2 küçük, 5 bir kaydırılır, çünkü 2'ye yer açması gerekir. Sonra 4 kaydırılır, sonra 3 kaydırılır. Kaydıracak eleman kalmayınca 2 başa konur: `2 3 4 5 1`. Aynı şekilde 1 için 5, 4, 3, 2 kaydırılır ve 1 başa konur: `1 2 3 4 5`.

5.3 Tasarım

Hoca tasarımı sorularla kurdu.

Soru: Eleman yerleştirme işlemine hangi elemandan başlanır, nereye kadar sürer? **Cevap:** Birinci indeksten başlar, $n-1$'e kadar sürer. İlk turda 1. eleman 0. ile karşılaştırılarak yerleştirilir, en son $n-1$. gözdeki eleman başa doğru taşınmaya çalışılır.

Yerleştirilecek eleman geçici bir değişkene (`temp`) alınır. Bu, "başa doğru taşıma niyetim olan sayı"dır. Başa doğru taşıma da bir döngüdür. Peki adım sayısı belli midir? Değildir, çünkü eleman yerini bulunca durur. Bu yüzden `while` kullanılır.

Taşıma en kötü ihtimalle nereye kadar sürer? En son 1. ile 0. eleman karşılaştırılır. Örneğin en küçük eleman 1 ise onu en başa kadar taşımak gerekir. `temp`'in ilk durduğu yer j. göz ise, `temp` $a[j-1]$ ile karşılaştırılır. En son $j-1$'in 0 olabilmesi için şartın $j > 0$ olması gerekir. `temp` küçükse $a[j-1]$ bir kaydırılır ve yola devam edilir.

- $i = 1$'den $n-1$'e kadar:
  - `temp = a[i]`, `j = i`.
  - `(j > 0) && (temp < a[j-1])` olduğu sürece: `a[j] = a[j-1]`, `j = j-1`.
  - Döngüden çıkınca `a[j] = temp`.

```c
#include <stdio.h>

void insertion_sort(int a[], int n) {
    int i, j, temp;
    for (i = 1; i <= n - 1; i++) {
        temp = a[i];
        j = i;
        while (j > 0 && temp < a[j - 1]) {
            a[j] = a[j - 1];
            j--;
        }
        a[j] = temp;
    }
}

int main(void) {
    int a[] = {3, 2, 8, 5, 9, 1};
    int i, n = sizeof(a) / sizeof(a[0]);
    insertion_sort(a, n);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Programın çıktısı `1 2 3 5 8 9` olur.

5.4 Ara Bir Adımı Elle İzleme

Baş tarafı sıralı bir dizide 3. gözdeki 4 yerleştirilecek. Dizinin 1. gözünde 5, 2. gözünde 7, 3. gözünde 4 vardır. $i = 3$ için `temp = 4`, $j = 3$.

j	Karşılaştırma	Sonuç	Yapılan
3	`temp` (4) < $a[2]$ (7)	evet	7, 3. göze kaydırılır, $j = 2$
2	`temp` (4) < $a[1]$ (5)	evet	5, 2. göze kaydırılır, $j = 1$
1	`temp` (4) < $a[0]$	hayır	döngüden çıkılır, `a[1] = 4`

5.5 En İyi ve En Kötü Durum

Dizi tersten sıralıysa çaresi yoktur, her elemanı tek tek başa taşımak zorunda kalır ve yine n^2 olur. Ama gerçek durumlarda genelde ortalama bir durumla karşılaşılır. Hepsini sondan başa taşımak gerekmez. Eleman yerini bulunca başlara bakmadan işi kesmesi, Insertion Sort'u Bubble Sort ve Selection Sort'tan avantajlı kılar.

Soru: Dizi küçükten büyüğe sıralıysa kaç karşılaştırma yapar? **Cevap:** $n - 1$. Çünkü `1 2 3 5 8` dizisinde 2, 1 ile karşılaştırılıp durur. 3, 2 ile; 5, 3 ile; 8, 5 ile karşılaştırılıp durur. Her seferinde döngü içinde bir kez karşılaştırılıp durulur. Dizi sıralıysa ya da sıralıya yakınsa işlem az sayıda karşılaştırmayla tamamlanır.

6. Shell Sort

6.1 Fikir

Insertion Sort'un büyük dizilerde bir sıkıntısı vardır: Başlarda olması gereken bir eleman sonlardaysa onu sondan başa taşımak zaman alır. Hocanın örneği: 1000 elemanlı bir dizide 900. gözdeki bir elemanı yavaş yavaş başa taşımak. Bunu hızlandırmak için Shell Sort geliştirilmiştir. Insertion Sort'a dayanır, ama elemanları öne hızlı taşımayı hedefler. Sınıfın bu algoritmayı daha önce görmediği ortaya çıkınca hoca "görmediğiniz bir sort çıktı sonunda" dedi.

Insertion Sort'ta her eleman kendinden öncekilerle karşılaştırılır. Shell Sort'ta ise bir adım büyüklüğü (step size) belirlenir. Adım büyüklüğü 3 ise her eleman kendinden 3 önceki gözdeki elemanla karşılaştırılır. Böylece dizi, her biri kendi içinde değerlendirilen parçalara ayrılmış gibi olur. Hedef, sonlarda kalmış küçük elemanları tek tek öne taşımak yerine "hoplata hoplata" taşımanın avantajından yararlanmaktır.

6.2 Adım Adım Örnek

Dizinin 6. gözünde önce 44 vardı. Hoca bu elemanın en başa kadar taşınışı görülsün diye onu 4 yaptı. Başlangıç dizisi ve adım büyüklüğü:

```
indeks:  0   1   2   3   4   5   6   7   8
dizi   : 54  26  93  17  77  31   4  55  20      step = 3
```

3'er atlanınca üç parça oluşur: 0, 3, 6. gözler (54, 17, 4), 1, 4, 7. gözler (26, 77, 55) ve 2, 5, 8. gözler (93, 31, 20). Step 3 olduğu için işe $i = 3$'ten başlanır.

Birinci parça:

- $i = 3$: 17, 3 önceki 54 ile karşılaştırılır. 17 küçük, 54 kaydırılır, 17 öne geçer.
- $i = 6$: 4, 54 ile karşılaştırılır, 54 kaydırılır. 4, 17 ile karşılaştırılır, 17 kaydırılır. 4 başa konur. Dizi: `4 26 93 17 77 31 54 55 20`.

İkinci parça:

- $i = 4$: 77, 1. gözdeki 26 ile karşılaştırılır, değişmez.
- $i = 7$: 55 daha küçük olduğu için 77 kaydırılır. 55, 26 ile karşılaştırılır, 55 büyük olduğu için 26 yerinde kalır. Dizi: `4 26 93 17 55 31 54 77 20`.

Üçüncü parça:

- $i = 5$: 31, 2. gözdeki 93 ile karşılaştırılır. 93 kaydırılır. 2. gözün 3 öncesi olmadığı için 31 oraya konur. Dizi: `4 26 31 17 55 93 54 77 20`.
- $i = 8$: 20, 5. gözdeki 93 ile karşılaştırılır, 93 kaydırılır. 20, 2. gözdeki 31 ile karşılaştırılır, 31 kaydırılır. 20, 2. göze konur. Dizi: `4 26 20 17 55 31 54 77 93`.

Hoca burada elde edileni gösterdi: Normal Insertion Sort ile 4'ü başa taşımaya kalksaydık tek tek karşılaştıra karşılaştıra taşıyacaktık. Şimdi 4 çok hızlı taşındı. 20 de hızlı bir şekilde başlara geldi.

Sonra step ikiye bölünür: $3/2$, tam sayı bölmesiyle 1 olur. Bu kez klasik Insertion Sort yapılır:

Eleman	Karşılaştırma ve kaydırma	Dizi
26	4 ile, değişmez	`4 26 20 17 55 31 54 77 93`
20	26 kaydırılır, 4 ile durur	`4 20 26 17 55 31 54 77 93`
17	26, 20 kaydırılır, 4 ile durur	`4 17 20 26 55 31 54 77 93`
55	26'dan büyük, değişmez	`4 17 20 26 55 31 54 77 93`
31	55 kaydırılır, 26 ile durur	`4 17 20 26 31 55 54 77 93`
54	55 kaydırılır, 31 ile durur	`4 17 20 26 31 54 55 77 93`
77	değişmez	`4 17 20 26 31 54 55 77 93`
93	değişmez	`4 17 20 26 31 54 55 77 93`

Hoca yukarıdaki diziyi i sırasına göre değil, parça parça izledi: $i = 3, 6$; sonra $i = 4, 7$; sonra $i = 5, 8$. Parçalar ortak göz kullanmadığı için bu sıranın sonuca etkisi yoktur.

> **Ders sonrası not:** Hoca izleme sırasında dizinin ara hallerini sözlü okurken bazı elemanları atladı ("4, 26, 93, 17, 55, 31, 54, 20" gibi). Yukarıdaki diziler, kodun çalıştırılmasıyla doğrulanmış tam ara hallerdir.
>
> **Ek bilgi (derste anlatılmadı):** Derste Shell Sort'un kodu yazılmadı. Aşağıdaki program, tahtadaki işlemin karşılığı olarak bu not için yazıldı. `gcc -Wall` ile derlenip çalıştırıldı. Tek farkı, Insertion Sort'taki 1 yerine `step` kullanması ve en dışta step'i ikiye bölen bir döngü olmasıdır.

```c
#include <stdio.h>

void shell_sort(int a[], int n, int step) {
    int i, j, temp;
    for (; step >= 1; step /= 2)
        for (i = step; i < n; i++) {
            temp = a[i];
            j = i;
            while (j >= step && temp < a[j - step]) {
                a[j] = a[j - step];
                j -= step;
            }
            a[j] = temp;
        }
}

int main(void) {
    int a[] = {54, 26, 93, 17, 77, 31, 4, 55, 20};
    int i, n = sizeof(a) / sizeof(a[0]);
    shell_sort(a, n, 3);
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Programın çıktısı `4 17 20 26 31 54 55 77 93` olur.

6.3 Hız ve Adım Büyüklüğünün Seçimi

Hoca, karmaşıklığın yine n^2 düzeyinde olduğunu ama Shell Sort'un Insertion Sort'tan daha hızlı olduğunu söyledi. Rastgele dizilmiş 100 bin sayıdan oluşan bir dizi üzerinde yapılan testte Shell Sort'un Insertion Sort'tan 600 kat hızlı olduğu bulunmuştur. Üstünlüğün sebebi, sonlarda kalan küçük elemanları büyük adımlarla öne taşımaktır.

Adım büyüklüğü dizinin büyüklüğüne bağlı olarak belirlenir. Genellikle $n/3$ civarından başlanır: 1000 eleman için önce 300 gibi, 100 eleman için 30 gibi. Sonra her seferinde ikiye bölünerek gidilir.

6.4 İkinci Gösterim

Hoca mantığı bir kez de yalnız sayılarla gösterdi. Step 3 ise 21, 8, 41 kendi içinde; 19, 36, 16 kendi içinde; 15, 22, 41 kendi içinde değerlendirilir. Dizi küçük olduğu için son adımda klasik Insertion Sort uygulanır. Böylece sonlardaki küçük elemanlar hızlıca başa taşınır.

Soru: Buradaki hız farkı step kadar, yani 3 kat mı olur? **Cevap:** Tam olarak öyle denemez, ortalama duruma göre değişebilir. Ama Insertion Sort'tan daha hızlıdır.

Soru: Step sayısını kullanıcı mı girer, yoksa dizinin eleman sayısına göre mi belirlenir? **Cevap:** Küçük dizilerde çalışırken baştan siz verirsiniz, sonra ikiye bölerek gidilir. Genelde step $n/3$ olarak alınır. Örneğin 100 elemanda 30'ar ilerlenir. Çok büyük alınmamalıdır.

Soru: Bu adımlarda parçaları neye göre sıralıyoruz, üç eleman arasında yine Insertion Sort mu yapıyoruz? **Cevap:** Evet. Kendi içinde bildiğiniz Insertion Sort yapılır. Hoca örnekle gösterdi: $i = 3$ için 3. gözdeki 21, 41'den küçük, 41 kaydırılır: `21 41`. 3 ilerlenir, 8 gelir. 41 ve 21 kaydırılır, 8 başa gelir: `8 21 41`. $i = 4$ için 19, 36'dan küçük, 36 kaydırılır, 19 öne gelir. 3 ilerisinde 16 vardır. 36 kaydırılır, 19 da kaydırılır (ara adım eklendi): `16 19 36`.

6.5 Adım Dizileri

Aradan sonra hoca, gördüğümüz dört algoritmanın karmaşıklıklarının hemen hemen aynı olduğunu, Shell Sort'un biraz daha iyi olduğunu tekrarladı. Klasik Shell Sort'ta artım $n/2, n/4, \ldots, 1$ biçimindedir. Yani step önce $n/2$, sonra $n/4$ olur ve böyle azala azala son adımda bildiğimiz Insertion Sort yapılır. Algoritma tasarımında duayen olan Knuth ise step büyüklüğünü 1, 4, 13 şeklinde giden bir artımla belirlemiştir.

> **Ders sonrası not:** Hoca 1000 elemanlı dizide klasik adımla ilk grubu "birinci, 500., 999." olarak söyledi. Step $n/2 = 500$ iken her grupta iki eleman vardır: 0. ile 500. gözler bir grup, 1. ile 501. gözler başka bir gruptur. Hocanın "ikinci, 501." devamı bu yapıya uyar; 999. göz ise 499. gözle aynı gruptadır.

Sonuçta Shell Sort, Insertion Sort'tan en az iki kat hızlıdır. Karmaşıklığı yaklaşık $n^{3/2}$'dir. Bu Insertion Sort'tan daha iyidir, ama küçük dizilerde çok tercih edilen algoritma yine Insertion Sort'tur.

> **Ders sonrası not:** "600 kat" ve "en az iki kat" ifadeleri çelişmez: Fark n büyüdükçe artar. Derste testin adım dizisi ve ölçütü söylenmedi. Bu not hazırlanırken 100 bin rastgele sayı üzerinde yapılan tek denemede karşılaştırma sayısı oranı klasik adımlarla yaklaşık 580, Knuth adımlarıyla yaklaşık 620 çıktı. Süre oranı ise yaklaşık 75-85 oldu. Yani 600 değeri bu denemede süreye değil, karşılaştırma sayısına yakındır.

7. Karmaşıklık Nedir?

7.1 Neden Süre Değil de n?

Hoca, karmaşıklığın önümüzdeki hafta daha ayrıntılı anlatılacağını, asıl analizlerin algoritma analizi konusunda yapılacağını söyleyerek başladı. Bir algoritmanın karmaşıklığı, girişte n eleman varken işlemin n'e bağlı olarak kaç kez yapılacağıdır.

Bir algoritmanın zamanı ne kadar iyi kullandığını söylemek için her yerde aynı anlaşılacak bir ölçü gerekir. "Şu işlemcili bir bilgisayarda 10 dakika çalışıyor" bilgisi size çok şey ifade etmez. Kendi algoritmanızı karşılaştırmak için tamamen aynı şartları sağlamanız gerekir ve bu mantıklı bir çözüm değildir. Herkes için ortak ölçü şudur: Girişte n eleman varken karmaşıklık hakkında n'e bağlı bilgi vermek.

7.2 En İyi, En Kötü ve Ortalama Durum

Değerlendirirken üç şeye bakılır:

- **En iyi durum (best case):** En önemsiz olanıdır. Karşınıza pek çıkmayacak bir durum olarak varsayılır. Dizi yarı sıralı olabilir, sırasız olabilir, ama her zaman küçükten büyüğe sıralı bir dizi beklenmez.

- **En kötü durum (worst case):** Genelde en çok dikkate alınan durumdur. “En kötü durumda bu algoritmanın karmaşıklığı ne olur?” sorusudur.
- **Ortalama durum (average case):** Sayıların ortalamasını almakla ilgisi yoktur. Rastgele giriş için beklenen değerdir, yani istatistiksel bir kavramdır.

Hoca bunu bir dizide eleman arama üzerinden açıkladı. En iyi durumda aranan eleman ilk elemandır. En kötü durumda dizinin son gözündedir. Ortalama durum ise elemanın ikinci, beşinci, dokuzuncu, yani herhangi bir gözde olabilmesi demektir. Dizi sıralarken de ortalama durum, dizinin ne küçükten büyüğe ne de büyükten küçüğe sıralı olduğu durumdur. Bazı elemanlar iki adımda yerine yerleşir, bazıları daha çok adımda.

Soru: En kötü durum mu daha önemli, ortalama durum mu? **Cevap (bir öğrenci):** Ortalama durum, çünkü daha sık gerçekleşir. Hoca pek çok çalışmada ortalama durumun daha önemli olduğunu, çünkü genel karşılaşılacak durumun bu olduğunu onayladı.

7.3 Planlama Örnekleri

Deprem ve telefon hatları. Deprem gibi önemli bir olay olduğunda herkes telefona sarılır, telefonlar kilitlenir ve kimse kimseye ulaşamaz. Hoca bunun sebebini sordu. Planlama ortalama duruma göre yapılmıştır: Örneğin aynı anda bir milyon insanın birbirini arayacağı varsayılmıştır, oysa o anda bütün Türkiye birbirini arıyordur. Santrallerde ve operatörlerde buna yönelik planlama yapılmamıştır. Neden? Bir öğrencinin dediği gibi, nadiren karşılaşılan bir durum için maliyeti yükseltmeye gerek görülmemiş olabilir. Hoca bunu “kesinlikle çok doğru” diye onayladı.

Üniversite sınav sonuçları. Sonuçlar açıklandığında ÖSYM’nin sitesine erişilememesi hakkında hoca öğrencilerin fikrini sordu. Öğrencilerden “en azından yılda iki kez olan bu durum için yapılmalıydı” ve “bu durumda en kötü durum artık ortalama gibi oluyor” cevapları geldi. Hocaya göre bu durum yılda iki kez olsa da çok önemlidir, herkes anında öğrenmek ister ve üstelik önceden bellidir. O yüzden burada en kötü duruma göre planlama yapmak daha doğru olabilir.

> **Hocanın vurgusu:** Algoritmanızın en iyi, en kötü ve ortalama durumlarını biliyorsunuz. En kötü durum ortalama durumdan farklıysa ihtiyacı düşünmelisiniz. Hangisine göre hareket edeceğinize ve önlem alacağınıza siz karar vermelisiniz.

8. Algoritmaların Karmaşıklık Hesabı

Hoca bu derste yalnız en iyi ve en kötü durumları hesaplayacağını söyledi.

8.1 Bubble Sort

En iyi durum. Dizi küçükten büyüğe sıralıdır, örneğin `1 4 5 9 11`. 1 ile 4, 4 ile 5, 5 ile 9, 9 ile 11 karşılaştırılır. 5 eleman için 4 karşılaştırma, genel olarak $n-1$ karşılaştırma yapılır.

> **Ders sonrası not:** Bu sonuç yalnız erken çıkışlı (`swapped` kontrollü) sürüm için geçerlidir. İlk tasarım dizi sıralı olsa da bütün turları döner ve $\frac{n(n-1)}{2}$ karşılaştırma yapar.

En kötü durum. Dizi tersten sıralıdır. Hoca küçük bir örnek gösterdi, `9 5 4 3`: $i = 1$ için baştan başlanır, ikişer karşılaştırılır: `5 9 4 3`, `5 4 9 3`, `5 4 3 9`. Yani $n-1$ karşılaştırma yapılır ve 9 yerleşir. $i = 2$ için $n-2$ karşılaştırma yapılır. Böyle gidilir, son turda 1 karşılaştırma yapılır. Toplam:

$$\sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} = \frac{n^2 - n}{2} \approx n^2$$

Hoca, $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ formülünü hatırlatarak buraya geldi. n sonsuza giderken 2’ye bölmenin önemi yoktur. Bubble Sort’un en kötü durum karmaşıklığı n^2, en iyi durum karmaşıklığı n’dir.

Ders sonrası not: Hoca toplamı önce $i = 1$'den $n-2$'ye kadar yazıp $\frac{(n-2)(n-1)}{2}$ buldu. Bir öğrencinin uyarısıyla düzeltti, doğrusu $n-1$'e kadardır. Hoca "son karşılaştırma $i = n-2$ için" dedi; ilk tasarımda tek karşılaştırma yapılan son tur $i = n-1$'dir. $n = 5$ için toplam $4+3+2+1 = 10$ eder ve formül $\frac{5 \cdot 4}{2} = 10$ verir.

Ders sonrası not: $\frac{n^2-n}{2} \approx n^2$ yazımı mertebe anlamındadır. Sayı olarak toplam, n büyüdükçe n^2'nin yarısına yaklaşır. 2 çarpanı karmaşıklığın mertebesini değiştirmediği için atılır.

8.2 Selection Sort

Önce $n-1$ karşılaştırmayla en küçük eleman bulunur ($i = 0$ için j, 1'den $n-1$'e gider). Sonra $i = 1$ için $n-2$ karşılaştırma yapılır. Böyle devam edilir ve en son 1 karşılaştırma yapılır. Toplam yine $\frac{n(n-1)}{2}$, yani yaklaşık $\frac{n^2}{2}$ olur.

Fakat burada en iyi durum en kötü duruma eşittir. Dizinin küçükten büyüğe mi, büyükten küçüğe mi sıralı olduğu konusunda elinizde bir bilgi yoktur. En iyi durumda da en kötü durumda da aynı karşılaştırmaları yapmak zorundasınız. Selection Sort'un karmaşıklığı her iki durumda da n^2'dir.

8.3 Insertion Sort

En iyi durum. Dizi sıralıdır. $n-1$ karşılaştırma yapılır. n sonsuza giderken 1 çıkarmanın önemi olmadığı için buna n denir.

En kötü durum. Dizi tersten sıralıdır, örneğin `5 4 3 2 1`, yani $a[0] > a[1] > \cdots > a[n-1]$. İç içe iki döngü vardır. Dış döngüde yaklaşık n yineleme (iteration) olur, en kötü durumda iç döngüde de yaklaşık n yineleme olur. i'nin değerlerine bakılırsa:

- $i = 1$ için 1 karşılaştırma: 4, 5 ile karşılaştırılıp yerleşir.
- $i = 2$ için 2 karşılaştırma: 3, önce 5 ile sonra 4 ile karşılaştırılır.
- Böyle gidilir, $i = n-1$ için $n-1$ karşılaştırma yapılır.

Selection Sort'ta i arttıkça karşılaştırma azalıyordu. Burada tersine artar. Toplam yine aynıdır:

$$\frac{(n-1)n}{2} = \frac{n^2}{2} - \frac{n}{2} \approx n^2$$

Ortalama durum. Hoca bunun hesabını Algoritma Analizi dersinde anlattığını söyledi. Sonuç yaklaşık $\frac{n^2}{4}$ mertebesindedir ve Bubble Sort ile Selection Sort'tan daha iyidir. Bu nedenle bu algoritmalar içinde tercih edilen Insertion Sort'tur.

Ders sonrası not: Bütün elemanları farklı rastgele bir dizide `temp < a[j-1]` karşılaştırmalarının beklenen sayısı $\frac{n^2+3n}{4} - H_n$'dir ($H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$). Bu formül, yukarıdaki kodla $n = 2$'den 8'e kadar bütün permütasyonlar sayılarak doğrulandı. Hocanın söylediği ifade büyük olasılıkla bu formülün $\frac{n^2}{4} + \frac{3n}{4} - \ldots$ biçimidir.

8.4 Shell Sort

Shell Sort'un karmaşıklığı artım büyüklüğüne (increment size), yani step size'a bağlı olarak değişir. Hoca en kötü durumda $n^{3/2}$'den başlayıp $n \log n$ ve $n \log^2 n$'e kadar giden karmaşıklıklar olduğunu söyledi.

Ders sonrası not: Bu değerler farklı adım dizilerine aittir. Klasik $n/2, n/4, \ldots$ dizisinin en kötü durumu n^2 mertebesindedir: n ikinin kuvvetiyken küçük yarı sıralı olarak tek gözlere, büyük yarı sıralı olarak çift gözlere konursa son adıma kadar hiçbir şey değişmez. Knuth dizisi ($h = 3h+1$: 1, 4, 13, 40, ...) en kötü durumda $O(n^{3/2})$ verir. $O(n \log^2 n)$ sonucu Pratt dizisine aittir.

8.5 Neden Bu Algoritmalar Büyük Dizilerde Kullanılmıyor?

n^2 karmaşıklıklı algoritmalar genelde büyük dizilerde kullanılmaz, çünkü n büyüdükçe çalışma süresi çok uzar. Küçük dizilerde ise ortalama durumda en verimlisi Insertion Sort olduğu için en çok o tercih edilir.

9. Karmaşıklığın Çalışma Süresine Etkisi

Hoca, karmaşıklıkların zamanı nasıl değiştirdiğini anlatmak için "çok sevdiği" bir tabloyu gösterdi. Insertion, Selection ve Bubble Sort n^2 karmaşıklıklıdır. Quick Sort, Merge Sort ve Heap Sort $n \log n$ karmaşıklıklıdır.

n	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0,003 µs	0,01 µs	0,033 µs	0,1 µs	1 µs	3,63 ms
10^8	0,027 µs	0,1 s	2,66 s	115,7 gün	-	-

$n = 10$ iken $n \log n$ algoritma 0,033 µs, n^2 algoritma 0,1 µs sürer. Hissedilecek bir fark yoktur. n büyütüldüğünde, örneğin 100 milyon olduğunda, $n \lg n$ 2,66 saniye sürer. Hoca n^2 için öğrencilere tahmin yaptırdı.

Soru: $n = 10^8$ için n^2 algoritma ne kadar sürer? **Cevap:** Öğrencilerden "dakika", "gün", "10 gün", "bir yıl" gibi tahminler geldi. Doğrusu 115,7 gündür. Aynı giriş için $n \lg n$ algoritmanın 2,66 saniye sürdüğü düşünülürse fark çok büyüktür.

$n!$ sütunu için $n = 10^8$ hesabı yapılmamıştır. Bu algoritma yalnız $n = 20$ iken bile 77,1 yıl sürer. Hoca bunu özellikle vurguladı: 1 milyon ya da 100 milyon değil, $n = 20$.

> **Ders sonrası not:** Tablo işlem başına 1 ns varsayımına uyar: $n^2 = 10^{16}$ ns $= 10^7$ s $= 115{,}7$ gün, $20! \approx 2{,}43 \times 10^{18}$ ns $= 77{,}1$ yıl. Hoca bazı değerleri farklı okudu: $n = 10$ için "0,33" (doğrusu 0,033 µs), mikrosaniye yerine "milisaniye", $n = 10^8$ için "0,07 mikrosaniye" (doğrusu 0,027 µs), "0,01 saniye" (doğrusu 0,1 s), "266 saniye" (doğrusu 2,66 s).
>
> **Ders sonrası not:** Kısa nottaki ekran okumasında $n = 10^8$ satırının 2^n sütununda 4×10^{13} yıl yazılıdır. Bu değer aslında $n = 100$ içindir: 2^{100} ns $\approx 4{,}0 \times 10^{13}$ yıl. $n = 10^8$ için 2^n çok daha büyüktür, bu yüzden yukarıdaki tabloda yazılmadı. Yine kısa notta $n = 10$, $n!$ için 3,66 ms yazar; $10! = 3\,628\,800$ ns $= 3{,}63$ ms'dir.
>
> **Hocanın vurgusu:** Bu, bir algoritma tasarlarken eleman sayınıza göre ne yapacağınıza karar vermek açısından gerçekten çok önemlidir. Tasarım yaparken ortamın şartlarının ne olacağını düşünerek çalışın.

Hocanın örneği üniversitedeki online sistemdi: Yaklaşık 20 bin öğrenci olduğunu hatırladığını söyledi. Bu sistemde hesap yaparken en kötü durum, 20 bin öğrencinin aynı anda bağlanıp ders izlemeye çalışması olabilir. Küçük bir iş yerinde ise aynı anda 100 kişi bağlanıyordur ve hesap ona göre yapılır. Şu anda dizilerle çalışılması, tasarımı öğrenmek içindir. Gerçek dünyada problemin şartlarına dikkat etmek gerekir, çünkü n^2 karmaşıklık 115 gün çalışmak anlamına gelebilir.

Hoca ikinci örnek olarak derin öğrenmeyi (deep learning) verdi. Bir sistemin eğitimi aylar sürebilir. Hocaya göre o algoritmalarda genelde optimizasyon düşünülmüyor, "ben vereyim, o öğrensin" mantığı var. Bir ya da on milyon örnek öğretmeye çalışıldığı için çok uzun sürmesi küçümsenecek bir şey değildir, ama daha verimli yapılarak bu süreler kısaltılabilir. Özellikle zamanın sıkıntılı olduğu durumlarda daha iyisini düşünmek her zaman faydalıdır.

Soru: Bu süreler CPU kullanılarak mı hesaplanmış? Derin öğrenmede ekran kartlarının gücü kullanılıyor. **Cevap:** Evet, bunlar çok eski kişisel bilgisayarlarda yapılmış hesaplar. Burada önemli olan, aynı donanım üzerinde n ve n^2 için makasın nasıl açıldığını görmektir. Hoca bunu güzel bir soru olarak nitelendirdi, çünkü karşılaştırmanın aynı şartlarda yapıldığının bilinmesi önemlidir.

10. Yerinde Sıralama (In-place Sorting)

Hoca, sıralamayla ilgili terminolojide kullanılan iki terimi sordu.

Soru: In-place sorting nedir? **Cevap:** Öğrencilerden "ana hafızada sıralama" ve "disk üzerinde sıralama" tahminlerinden sonra "yeni bir dizi yaratmadan" cevabı geldi. Hoca bunu onayladı: Dizinin üzerinde, yedek dizi kullanmadan yapılan sıralamalardır.

Selection Sort, Quick Sort ve Insertion Sort yedek dizi kullanmadan aynı dizi üzerinde sıralama yapar. Merge Sort in-place değildir. Sıralama yapabilmek için ekstra yer ister, yedek dizi kullanır.

Soru (bir öğrenci): Bu biraz hız mı, yer mi ödünleşimi (trade-off) değil mi? **Cevap:** Evet. Genellikle in-place yapılmaya çalışılır, çünkü problemleri hep eleman sayısının fazla olduğu durumlar için düşünmeliyiz. Eleman sayısı fazlayken bir de yedek yer kullanmak iyi bir şey değildir.

Yine de Merge Sort, $n \log n$ karmaşıklıklı algoritmalar içinde tercih edilen bir algoritmadır. Quick Sort'un en kötü durum karmaşıklığı n^2'dir. Merge Sort'ta ise en kötü, ortalama ve en iyi durumların hepsi $n \log n$'dir. Bu yüzden şartlara göre ekstra yer tolere edilebilir. Karar tamamen probleme bağlıdır. Genel olarak yedek dizi kullanılması yalnız sıralamada değil, pek çok algoritmada tercih edilmez. Yapılabiliyorsa dizinin kendi üzerinde yapılması tercih edilir.

11. Kararlı Sıralama (Stable Sorting)

Hocanın vurgusu: Bu kavram önemlidir, çünkü gerçekten kullanılır.

Soru: Stabil sıralama nedir? Karmaşıklığın aşağı yukarı aynı kalması mı? **Cevap:** Hayır. Hoca kavramı bir örnekle anlattı.

Kişiler önce isimlerine göre sıralanmış olsun. Sonra doğum yılına göre sıralanmak istensin. Aslında doğum yılına göre sıralı bir liste istiyoruz, ama doğum yılına bakarken isimlerin sırası bozulmasın.

İsme göre sıralı	Doğum yılı
Ali	1999
Ayşe	2001
Banu	1998
Berna	1999
Canan	2001

Kararlı bir sıralamayla doğum yılına göre sıralanınca sonuç şudur: Banu (1998), Ali (1999), Berna (1999), Ayşe (2001), Canan (2001). Banu en üste gelir, ama 1999 doğumluların kendi içindeki sırası bozulmaz. İlk sıralamada Ali, Berna'dan yukarıdaydı, ikinci sıralamada da yukarıda kalır. Berna Ali'nin üstüne çıkmaz. Aynı şekilde Canan da Ayşe'den sonra yer alır. Yani ikinci sıralamayı yaparken ilk sıralamadaki eşit değerler için ilk sıra bozulmaz. Excel'de de bu görülür: Öğrencileri notlarına göre sıralayıp ikinci olarak numaralarına göre sıralarsınız. Notlara göre sıralı olur, numaralar da kendi içinde sıralı olur.

Soru: Doğum yılı aynıysa ilk sıraya göre mi değişiyor? **Cevap:** İkinci sıralamada, ilk sıralamadaki eşit değerler için ilk sıra bozulmaz. 1999'luların içinde Ali, Berna'dan önce geliyordu, yine önce gelir.

Soru: Hangi algoritmalar kararlıdır?

Insertion Sort kararlıdır. Hoca bunu örnek üzerinde gösterdi. Ali 1999, Ayşe 2001, Banu 1998 ile başlayalım. 2001, 1999'dan küçük mü? Hayır, yerinde kalır. 1998, 2001'den küçük mü? Evet. Banu bütün olarak öne taşınır. Banu'nun arkasında bir de Ferda (1998) olsun. Ferda'yı taşırken o, kendinden öncekilerle karşılaştırılır. Banu ile eşit olduğu için Banu'dan önceye geçemez, arkasında kalır. Sıra bozulmaz.

Bubble Sort kararlıdır. Bir öğrenci bir soru sordu.

Soru: En sona gelmesi gereken bir eleman en baştaysa bütün elemanlarla yer değiştirerek gitmeyecek mi? Bu nasıl stabil olabilir? Örneğin Canan (2001) en başta olsa bütün 1999'luları geçmeyecek mi? **Cevap:** Geçer, ama bunda bir sıkıntı yoktur, çünkü sıralama zaten bunu gerektirir. Önemli olan, eşit değerdeki elemanların sırasıdır. Diziye bir de Demet (2001) eklensin. Canan, Demet'e geldiğinde ikisi de 2001 olduğu için yer değiştirmez. Canan orada kalır, Demet'in önündeki sıra bozulmaz.

Selection Sort kararsızdır (unstable). Bir öğrencinin cevabı da buydu: Eşitler arasında kıyaslama yapmadığı için hangisine denk geldiyse onu taşır. Hoca bunu onayladı: Hiçbir şeye bakmadan elemanı yukarı taşıyıverir.

> **Ek bilgi (derste anlatılmadı):** Selection Sort'un kararsızlığı, uzaktan yapılan yer değiştirmeden kaynaklanır. $a[i]$ ile $a[adr]$ yer değiştirirken $a[i]$ sağa sıçrar ve arada kendisine eşit bir eleman varsa onun gerisine düşer. Böylece eşitlerin ilk sırası bozulur. Insertion Sort ve Bubble Sort'ta ise elemanlar yalnız komşu gözler arasında hareket eder ve eşitler yer değiştirmez.

Quick Sort da kararsızdır. Hoca, onu henüz görmediğimizi ama kararsız olduğunu ekledi.

Bu özellik şu nedenle önemlidir: Bir şeyi birinci şarta göre sıraladınız. İkinci şarta göre sıralarken birincinin kendi içindeki sırasının bozulmasını istemiyorsanız, hangi algoritmayı kullanacağınıza buna göre karar vermelisiniz.

Dersin Toparlaması

Bu derste Bubble, Selection, Insertion ve Shell Sort, yalnız nasıl çalıştıklarıyla değil, birbirini nasıl iyileştirdikleriyle ele alındı. Bubble Sort'a "iş bitince dur" kontrolü eklendi. Selection Sort yer değiştirmeyi azalttı, ama durma şansını kaybetti. Insertion Sort elemanın yerini bulunca durarak ikisinin de avantajını yakaladı ve küçük dizilerin tercih edilen algoritması oldu. Shell Sort, sonlardaki küçük elemanları büyük adımlarla taşıyarak Insertion Sort'un büyük dizilerdeki sıkıntısını hafifletti. Karmaşıklık, makineden bağımsız olarak n'e göre ifade edildi ve tablo, n^2 ile $n \log n$ arasındaki makasın büyük n'de nasıl açıldığını gösterdi. Yerinde ve kararlı sıralama kavramları da algoritma seçiminde kullanılacak iki ölçüt olarak eklendi.

Hoca, Shell Sort'u hazır çözümlere bakmadan kendi başına yazmayı önerdi. Bu sıralama dersleri hatırlatma ve dengeleme amacıyla planlanmıştır. Merge Sort ve Quick Sort, ilerleyen haftalarda böl ve yönet algoritmalarıyla birlikte asıl konu olarak anlatılacak. Karmaşıklık analizi de önümüzdeki derslerde daha ayrıntılı ele alınacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.